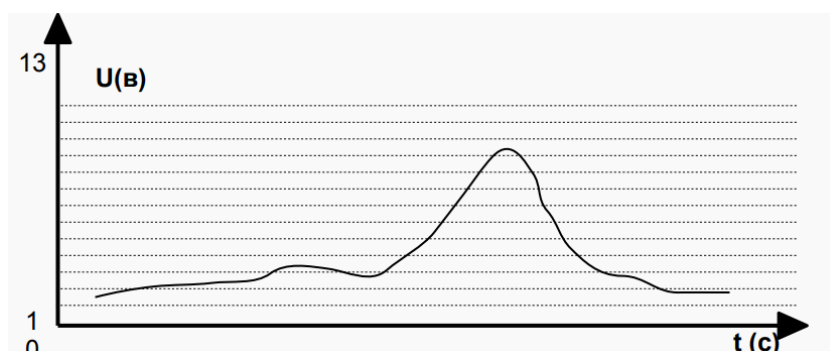


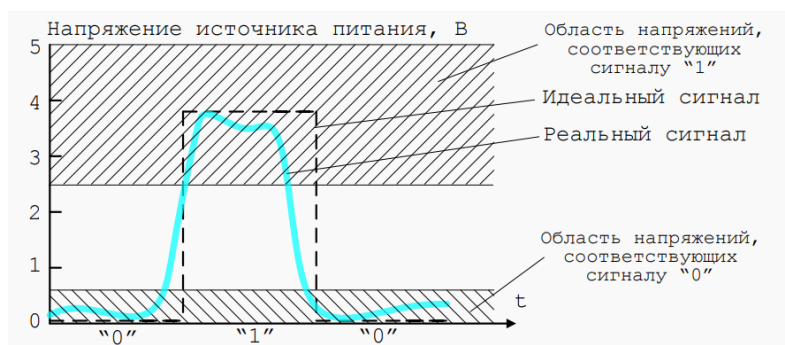
1) Две формы представления информации. Способы представления дискретной информации. Системы счисления, используемые в вычислительной технике: двоичная, 8-я, 10-я, 16-я, двоично-десятичная.

Формы представления информации:

Аналоговое представление (аналоговые ЭВМ) – имеют высокое быстродействие, меньшая погрешность вычислений. Реальным параметрам ставиться в аналог напряжение, величины непрерывны, изменяются по одному и тому же закону. Для сложения аналоговых величин достаточно соединить два провода в один. Маленькая погрешность вычислений. Обладает большими габаритами.



Дискретное представление (цифровые ЭВМ) – представляет информацию с помощью только двух дискретных величин – 0 и 1. 1 – высокое напряжение, 0 – низкое напряжение.



С помощью дискретных величин можно создать логические элементы ЭВМ

• Логические элементы

ГОСТ	ANSI	ГОСТ	ANSI
Буфер	BUF	ИЛИ	OR
Инвертор	INV	ИЛИ-НЕ	NOR
И	AND	ИСКЛЮЧАЮЩЕГО ИЛИ	XOR
И-НЕ	NAND	ИСКЛЮЧАЮЩЕГО ИЛИ-НЕ	XNOR

И				
X1	X2	Y	Y	Y
0	0	0	1	
0	1	0	1	
1	0	0	1	
1	1	1	0	

ИЛИ				
X1	X2	Y	Y	Y
0	0	0	1	
0	1	1	0	
1	0	1	0	
1	1	1	0	

Искл. ИЛИ				
X1	X2	Y	Y	Y
0	0	0	1	
0	1	1	0	
1	0	1	0	
1	1	0	1	

С помощью дискретной информации можно представлять информацию в виде последовательности нулей и единиц. В вычислительной технике используются 2-я, 8-я, 10-я, 16-я, двоичная-десятичная.

Двоичная система счисления используется для представления любых данных в процессоре, оперативной памяти. 0 отвечает за то, что тока нет, 1 отвечает за то, что ток есть. Занимает ровно 1 бит.

Восьмеричная система счисления применялась, как компактная запись двоичных кодов (в старых ЭВМ, UNIX правах). Занимает ровно 3 бита.

Шестнадцатеричная система счисления используется, как компактная запись двоичных кодов. Используется в БЭВМ, в записи цветов 0xFFFFFF. Занимает ровно 4 бита (тетраду).

Десятичная система счисления – понятная, привычная и приятная для восприятия человека система счисления.

Двоично-десятичная система счисления – каждая цифра десятичного числа кодируется 4 битами двоичной системы счисления. Например, 19 = 0001 1001. Используется в финансовых и бухгалтерских расчётах, где важна точность вычисления.

2) Представление чисел с фиксированной точкой. Прямой, обратный и дополнительный код. Формирование битовых признаков переноса, переполнения, отрицательного результата, нуля.

3) Представление символьных и строковых данных. Принципы построения кодовых таблиц ASCII, КОИ-8, ISO8859-5, Windows-1251, UTF-8, UTF-16.

4) Базовые элементы вычислительной техники: ячейки, регистры, шины, вентили, тактовые генераторы, логические схемы, триггеры, регистры, счетчики, сумматоры.

5) Структура и принцип функционирования ЭВМ. Порядок функционирования простого процессора на примере калькулятора.

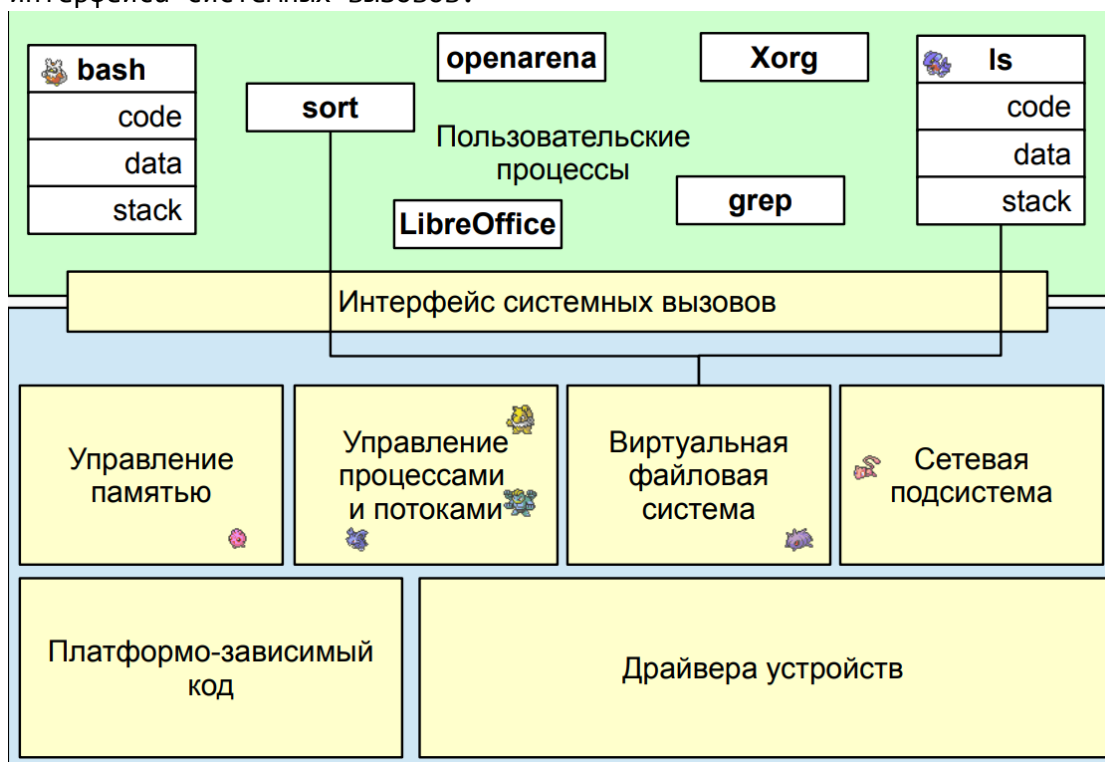
6) Операционная система Unix – ядро ОС и файловая система

Историческое предназначение для замены работы оператора компьютерной системы. Оператор – человек, который управлял запуском и настройкой программ на вычислительной машине. Оператор выполнял функции интерфейса, файловой системы, выдачи процесса.

Делятся операционные системы на **пользовательские, серверные и встроенные ОС**. Пользовательские имеют удобный интерфейс, принцип диспетчеризации задач (задачи выполняются в заданное время), а встроенные нет. Серверные операционные системы распределяют ресурсы между запросами. Встроенные операционные системы работают по принципу реального времени.

Linux использует те же системные вызовы, что и Unix. При этом у Unix другое ядро.

Ядро *NIX. Содержит пользовательские пакеты, интерфейс системных вызовов, системные утилиты для работы ядро. code, data, stack – специальные адреса данных кода и стека (служит для возврата назад в программы, лежат в файловой системе). Пользователь действует с приложением -> приложение запрашивает системные вызов -> управление процессами, памятью, потоками ловит вызов и отвечает. Ядро является посредником между железом и пользовательским интерфейсом. Пользовательские процессы взаимодействуют с ядром с помощью интерфейса системных вызовов.

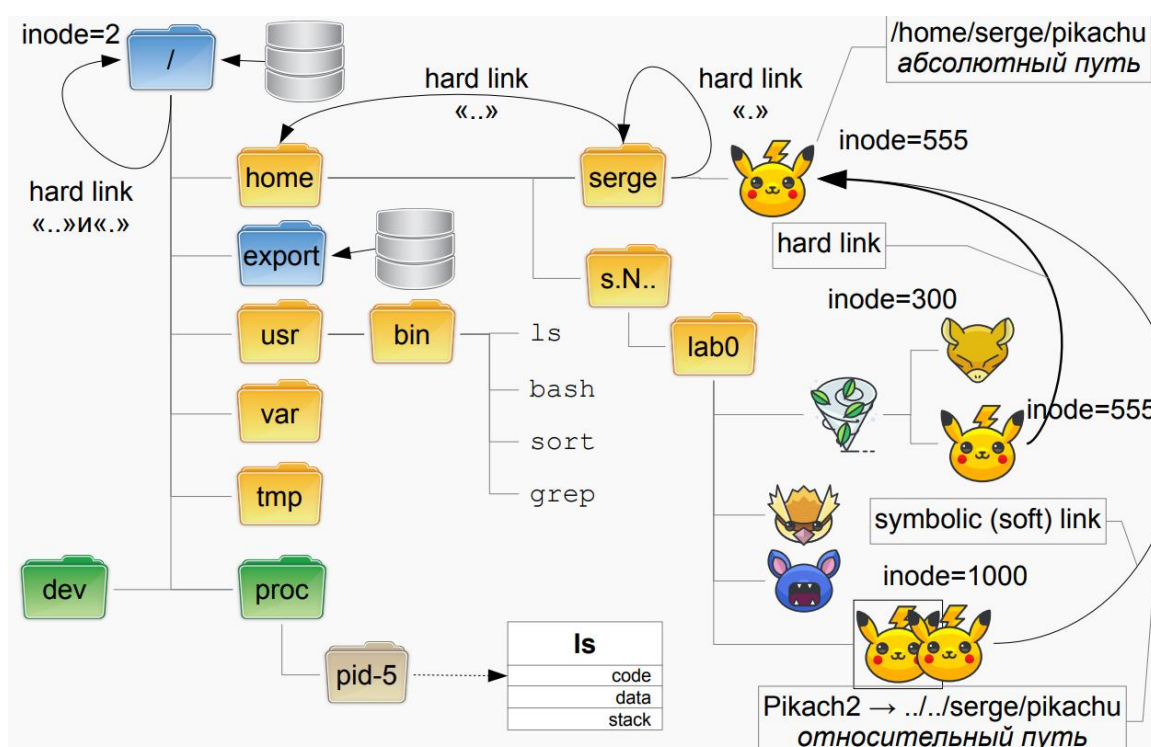


Системные вызовы стандартны в системах *NIX. Системные вызовы Windows похожи на системные вызовы *NIX.

Платформено-зависимый код - отвечает за запуск на разных моделях процессоров и тд.

Виртуальная память – есть в файловой системе. Программе выдаются страницы памяти, эти страницы виртуальные, и могут быть как на жестком диске, так и на оперативной памяти зависит от состояния программы. Может расширяться за счёт

Относительный путь начинается от `./` (`./work/lab8`)



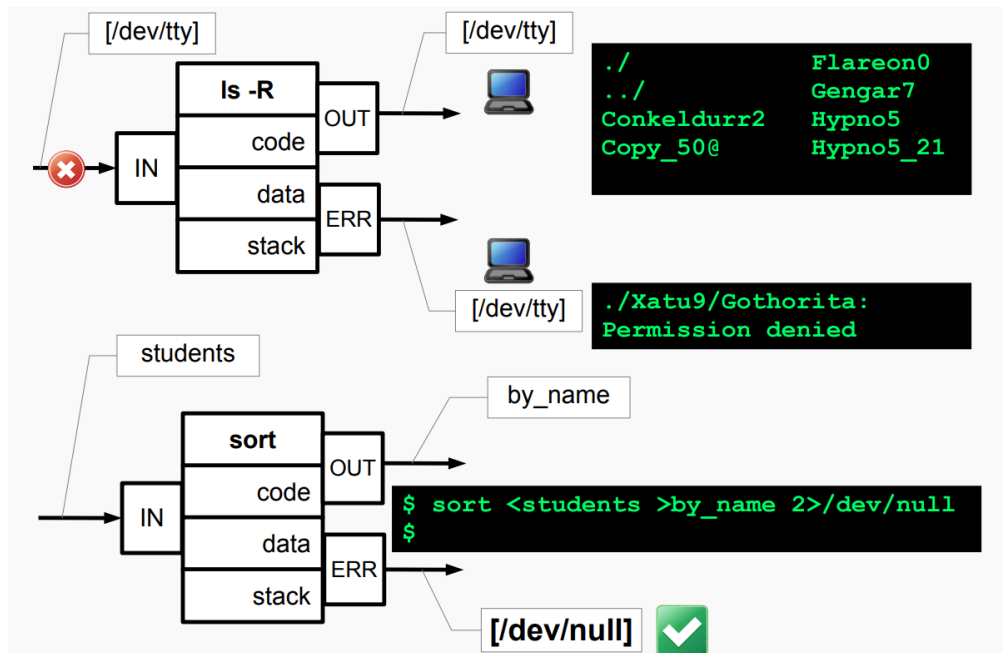
7) Операционная система Unix – интерпретаторы, стандартные потоки ввода вывода, фильтры.

Потоки `stdin(0)`, `stdout(1)`, `stderr(2)`

`stdin(0)` – поток ввода

`stdout(1)` – поток вывода

`stderr(2)` – поток ошибок



По умолчанию вывод идёт в терминал.

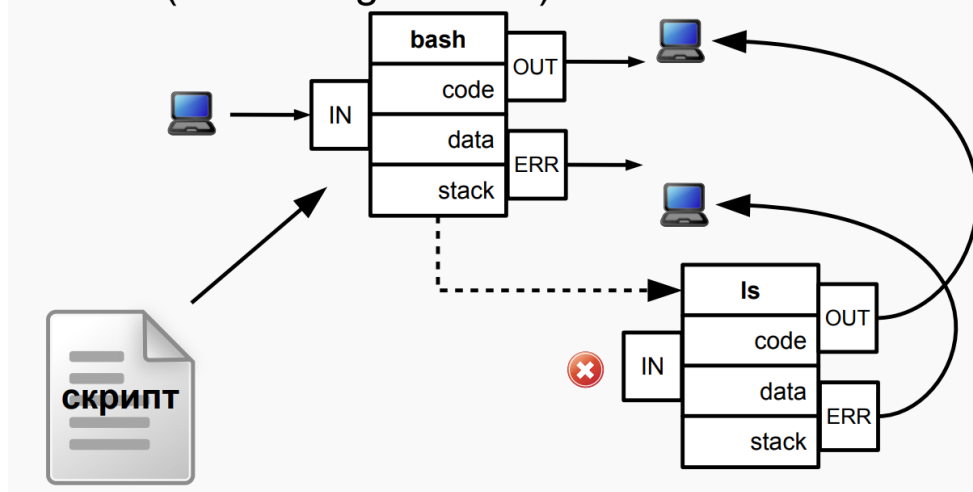
> - перенаправление потока ввода

< - перенаправление потока вывода

Интерпретатор команд – программа, которая осуществляет ввод вывод результата соответствующей инструкции (команды), по умолчанию в терминал.

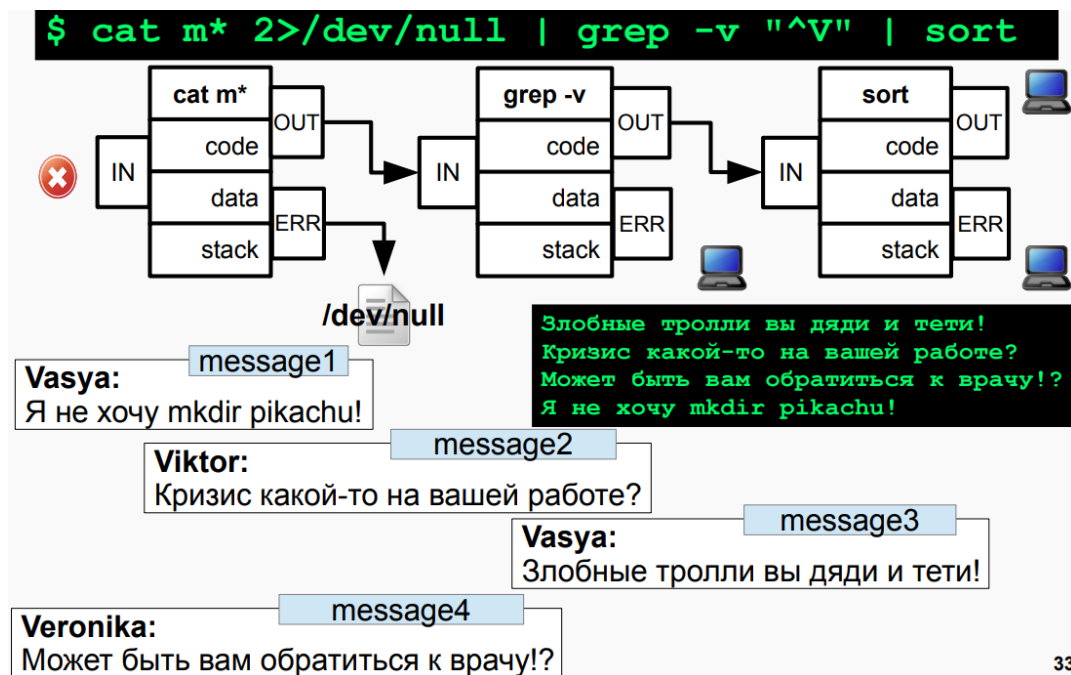
Популярные интерпретаторы `sh` (bourne shell), `ksh` (korn shell), `csh` (c shell), `bash` (bourne-again shell)

- `sh` (Bourne shell) `ksh` (Korn shell) `csh` (C shell)
`bash` (Bourne-again shell)



- `> file` — перенаправить stdout в `file`
- `>> file` — добавить stdout к `file`
- `2> file` — перенаправить stderr в `file`
- `2>> file` — добавить stderr к `file`
- `< file` — взять stdin из `file`
- `<< EOF` — записать в stdin из терминала до символов «EOF»
- `ls | sort` — перенаправить stdout команды `ls` на stdin команды `sort`

Фильтры происходят с помощью команды `grep`. В ней можно задавать регулярные выражения



messages

Vasya:
Злобные тролли вы дяди и тети!

Viktor:
Кризис какой-то на вашей работе?

Veronika:
Может быть вам обратиться к врачу!?

Vasya:
Я не хочу mkdir pikachu!

- Символ — соотв. сам себе
- `^` - начало строки
- `$` - конец строки
- `.` - 1 любой символ

```
$ grep mkdir messages
Я не хочу mkdir pikachu!
$ grep "^V" messages
Vasya:
Viktor:
Veronika:
Vasya:
$ grep "т..$" messages
```

34

8) Операционная система Unix – основные команды, права файлов и способы их задания

Файл содержит, имя, количество жёстких ссылок, права на этот файл, директория ли он или нет, владельца, группу владельца, дату модификации, последнего доступа [-], изменения inode [-c]. Выводится с помощью команды `ls -la`.

R - read, W - write, X - execute

[d] [r][w][-] [r][-][-] [r][-][-] пример прав для директории

[-] [r][w][-] [r][w][x] [r][w][-] пример прав для файла

0 символ отвечает за тип файла

1-3 отвечают за права владельца

4-6 отвечают за права группы

7-9 отвечают за права остальных

Кол-во жестких ссылок

Владелец

Группа-владелец

Имя

```
s207549@helios:/export/home/studs/s207549/lab0$ ls -la
total 26
drwxr-xr-x  5 s207549  studs   10 дек.  8 2015 ./
drwxr-xr-x 24 s207549  studs   37 дек.  8 2015 ../
----rw----  1 s207549  studs   21 дек.  6 2015 Conkeldurr2
lrwxrwxrwx  1 s207549  studs    5 дек.  6 2015 Copy_50 -> Xatu9/
dr-x--x-wx  5 s207549  studs    9 дек.  8 2015 Flareon0/
drwx-wxrw-  4 s207549  studs    8 дек.  8 2015 Gengar7/
-rw-----  2 s207549  studs   37 дек.  6 2015 Hypno5
-rw-r--r--  1 s207549  studs  183 дек.  8 2015 Hypno5_21
-r--r----- 1 s207549  studs  285 дек.  6 2015 Psyduck4
dr-xr-xr-x  5 s207549  studs    8 дек.  8 2015 Xatu9/
```

Тип файла

Права для владельца

Права для группы

Права для остальных

Read, Write, eXecute

Directory, sybolic Link, file (-)

Дата:

- модификации
- последнего доступа [-u]
- изменения inode [-c]

Способы задания:

Файлы задаются с помощью команды `chmod 007`. Удобно задавать так и user, g group, o other. Выбираем какие права хотим дать кому-то, например user хотим дать r-x, соответствует 101 -> 5 в десятичной системе. Тогда можем задать права для пользователя `chmod 500`. Для группы и остальных делается аналогично.

Тип файла

Права для владельца

Права для группы

Права для остальных

1 1 0 1 0 0 1 0 0 → = 644

u+x g+rx o=x

r w x r - x - - x

1 1 1 1 0 1 0 0 1 → = 751

Файлы могут быть доступны человеку из какой-либо группы.

Основные команды:

mkdir – создаёт директорию
echo – печатает текст в файл или в терминал
cat – объединяет файлы и выводит их содержимое в поток вывода
touch – “трогает”, создаёт файл
ls – выводит содержимое о каталоге
pwd – выводит текущее местоположение в файловой системы
cd – переходит по пути (относительному/абсолютному)
more – выводит содержимое файла по страницам
cp – копирует файл один в файл два `cp [f1] -> [f2]`
rm – удаляет файл (`rm -R` удаляет рекурсивно)
rmdir – удаляет директорию
mv – переместить файл
head – выводит первые 4 строки
tail – выводит последние 4 строки
wc – выводит количество слов, байтов и тд в файле
sort – сортирует файл, или содержимое
grep – фильтрация, позволяет писать регулярные выражения
man – справка по команде
| – конвейер, передает результат исполнения слева на вход вправо

Команда	Назначение и синтаксис
mkdir	mkdir [-m mode] [-p] dir...
echo	echo [string]...
cat	cat [-n] [file...] [-]
touch	touch [-am]... file...
ls	ls [options] [file/dir]...
pwd	pwd
cd	cd [argument]
more	more [file...]
cp	cp [options] SOURCE ... DEST
rm	rm [options] [file/dir]
rmdir	rmdir [dir]
mv	mv [-fi] SOURCE ... DEST
head	head [-num] [file...]
tail	tail [-/+num] [-bcl] [file...]
sort	sort [-unr] [-k num] [file...]
grep	grep [-v] regexp [file...]
wc	wc [-c -m] [-lw] [file...]

messages
 Vasya:
 Злобные тролли вы дяди и тети!
 Viktor:
 Кризис какой-то на вашей работе?
 Veronika:
 Может быть вам обратиться к врачу!?
 Vasya:
 Я не хочу mkdir pikachu!

- Символ — соотв. сам себе
- ^ - начало строки
- \$- конец строки
- . - 1 любой символ

```

$ grep mkdir messages
Я не хочу mkdir pikachu!
$ grep "^V" messages
Vasya:
Viktor:
Veronika:
Vasya:
$ grep "т..$" messages
  
```


9) Состав и структура БЭВМ. Адресные пространства БЭВМ. Система команд БЭВМ, форматы команд. Машинные циклы.